

Kev and CLM: two ways to decide

Difference	Kev (small models)	CLM
Inputs	State, question and options together	State and actions encoded separately
Trainable	LoRA + pointer head	Projection heads only
Features	Encoder behavior changes	LLM features stay fixed

Our lab follows Kev's LoRA + pointer-head recipe.

A CLM decision, step by step

01 Inputs

CLM-8B teaching illustration

STATE + QUESTION

My invoice was charged twice.
Which team should handle this?

CANDIDATES SUPPLIED BY CODE

1. Charges, invoices, refunds
2. Bugs and outages
3. Product purchases

→
Separate texts enter CLM

Input: state s and supplied candidates a_1, a_2, a_3

s : state plus question/instructions; a_j : candidate j ; j : candidate index.

K : number of supplied candidates. Code or a separate LLM supplies them.

Training and serving cost

Work	Key LoRA	CLM heads
Training	Encoder forward + backward for adapters and head	Extract fixed features then train two heads
Repeat runs	Repeat encoder compute for every experiment	Reuse feature cache Train heads only
Serving	Encode state and options Reuse a shared prefix	Encode changing state Reuse action vectors

Total cost = feature extraction + training + serving.

Performance: compare like with like

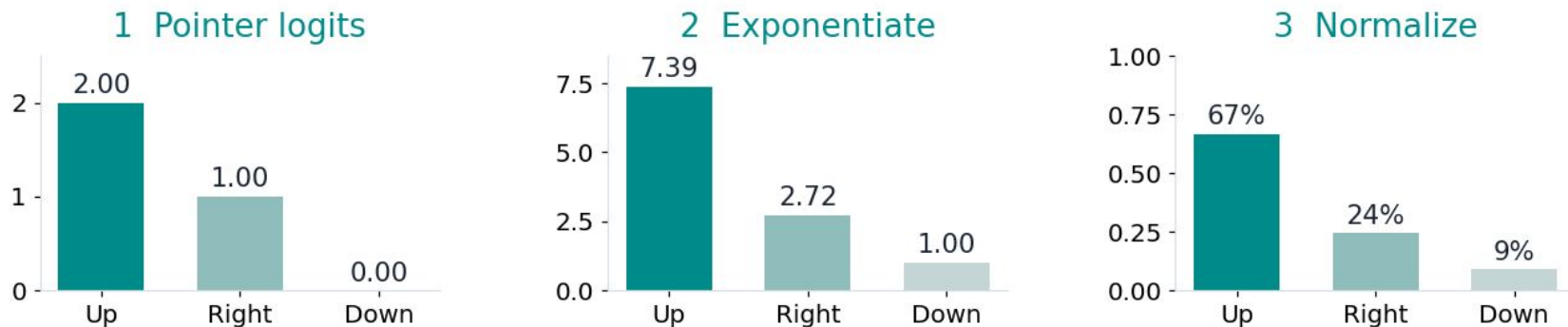
Published result	Setup	What it supports
CLM: 36-44 ms	RTX 4090; cached actions 1-1,024 candidates	Low added cost as the action set grows
Kev-4B: 12 ms	RTX PRO 6000 One question; median	Fast serving in that llama.cpp setup
Kev vs CLM	No matched Pac-Man comparison established	Measure skill, latency and memory together

Different hardware and tasks: these are not a head-to-head result.

Serving: models and readouts

Model	Readout	Useful strength	Main tradeoff
Julia-1 144M	Option-marker head	Small; multilingual 50+ languages	Limited knowledge/reasoning Private training pipeline
Laya 421M	Option-marker head	Small; designed for specialization	Fully fine-tuned encoder English root: 512 tokens
Kev-4B	Pointer head	Adaptable features Open training code	LoRA changes features Options encoded each step
lev 4B	Vocabulary label logits	Existing LM readout Many label codes	LoRA; two choice orders Extra head not converted
OpenJev 27B	Vocabulary label logits	Text and images Agent decisions	Larger model Non-commercial weights

Key scores the legal moves



Illustrative logits [2, 1, 0]; three legal moves. These are not trained-model measurements.

$$z_j = \frac{q^\top k_j}{\sqrt{d_p}} \quad p_j = \frac{e^{z_j}}{\sum_{k \in \mathcal{A}(s)} e^{z_k}}$$

q : decision vector; k_j : option vector; $d_p = 256$; z_j : logit

p_j : probability; $\mathcal{A}(s)$: supplied legal moves; raw temperature = 1

[Interactive scoring controls](#) | Illustrative probabilities; fresh checkpoints remain uncalibrated.

Our 90-minute route

0–20 Compare models, animation and play

20–30 Review Pac-Man planning labels

30–60 Fine-tune LoRA and the pointer head

60–80 Evaluate the same boards and seed

80–90 Explain and submit CP1

Prepare Colab before class

RTX PRO 6000 Blackwell: 96 GB target

Pinned PyTorch/CUDA; BF16 autocast

Optimized FLA, convolution and fused AdamW

Pac-Man: batch 4 × accumulation 2

Checkpointing on; row budget 0

Five stages, five separate adapters

- 1 Decisions: 12,576 records $\times$ 2 epochs
- 2 Dates/evidence: 1,425 + 2,000 replay
- 3 Documents: 5,219 + 2,000 replay
- 4 Skills/devtools: 11,320 + 4,000 replay
- 5 Pac-Man: 4,096 + 2,000 replay; 1 epoch

Interactive play: meet the four ghosts

Four native ghost personalities and release rules

Remaining power pellets: 0 in the maze

After consumption: global + per-ghost frightened flags

Badge: Skills LoRA before; Pac-Man LoRA after

Planning labels from native game states

4,096 train; 256 dev; 256 evaluation

Search 20 moves, beam 8; rank survival first

1,450 training boards have power mode active

$$J = \Delta S + 5P - 4R - 6D_p + 2\min(D_g, 8)$$

J: path value; ΔS : score gain; P: pellets; R: revisits

D_p : nearest-pellet distance; D_g : distance to a dangerous ghost

CP1: fine-tune and evaluate Pac-Man

Skills → Pac-Man: rank-16 LoRA + pointer head

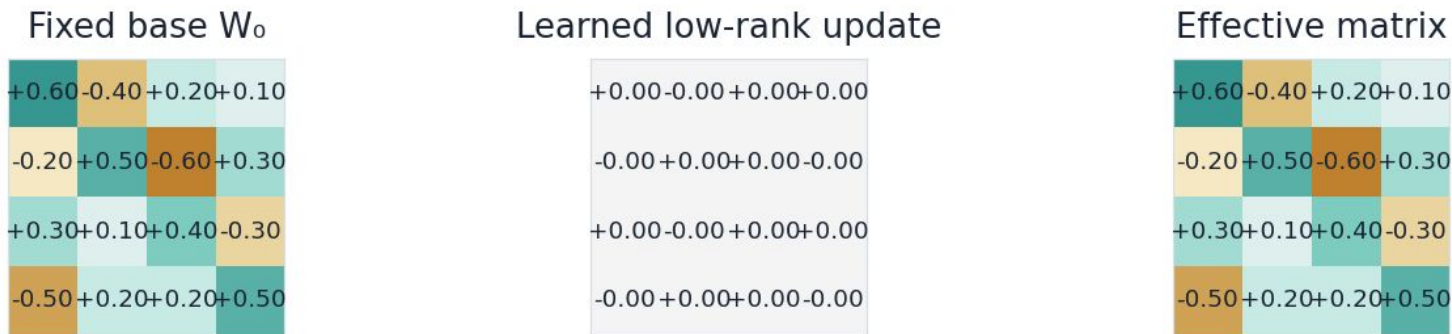
4,096 boards + 2,000 replay; one epoch

762 updates at learning rate 2e-5

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B} \sum_{i=1}^B \log p(y_i | s_i)$$

B: batch size; s_i: board; y_i: teacher move; p(y_i | s_i): predicted share

LoRA updates and training evidence



Illustrative 4×4 matrices, rank 2, $\alpha/r = 1$. Kev uses rank 16 and $\alpha = 32$.

$$W_{\text{eff}} = W_0 + \frac{\alpha}{r}BA$$

W_0 : fixed base; A : $r \times d_{\text{in}}$; B : $d_{\text{out}} \times r$

$r = 16$; $\alpha = 32$; 33.8M trainable LoRA/head parameters

[Interactive LoRA controls](#) | TensorBoard + recovery; verified Drive backup after each saved snapshot.

Evaluate decisions and gameplay

Same 256 boards: strict and tie-aware agreement

Search regret, survival choices and immediate captures

Same-seed gameplay: pellets, score and revisits

$$A = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{a}_i = y_i] \quad \hat{a}_i = \arg \max_j p(j | s_i)$$

$N = 256$; y_i : teacher move; $\hat{a}_i$: model move; $\mathbf{1}[\cdot]$: agreement indicator

Submit CP1: show what changed

Explain a changed move or remaining mistake

Connect the move to the ghost behaviour

Show trainable parameters and training curves

Save task adapter and paired evaluation results

Submit the notebook, runtime and compute use